Reviewer Pack — Minimal Order of Automorphism Groups in Symplectic Geometry …

Entry b0d16ff55b01 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 12:21:10 UTC

Minimal Order of Automorphism Groups in Symplectic Geometry Bounds Communication Complexity

Entry ID: `b0d16ff55b01` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 12:21:10 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Communication Complexity

Statement:

> The minimal order of automorphism groups for a symplectic manifold M bounds the deterministic communication complexity of evaluating a polynomial f on M, such that for all polynomials f and all symplectic manifolds M with automorphism group order ≤ n, E[communication_complexity(f, M)] = O(n^2).

Rationale (proposer's reasoning):

> Symplectic geometry has a rich algebraic structure that may encode nontrivial properties that can influence the complexity of communication tasks. The minimal order of automorphism groups captures the combinatorial complexity of symplectic structures, which could potentially relate to the complexity of information exchange in distributed computations.

Taxonomy category: `SymplecticGeometry` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `08478111b1375c79`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For symplectic manifold M with automorphism group order ≤ n, if the expected communication complexity E[communication_complexity(f, M)] is within ±10% of O(n^2), then support the conjecture; otherwise, falsify it.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `symplectic geometry AND communication complexity AND polynomial evaluation` - `automorphism groups in symplectic geometry AND deterministic communication complexity` - `minimal order bounds AND communication complexity of polynomials on symplectic manifolds`

Top relevant hits considered: - [http://arxiv.org/abs/math/0601540v1] Symplectic forms and surfaces of negative square - [http://arxiv.org/abs/1811.09984v4] Translated points for contactomorphisms of prequantization spaces over monotone symplectic toric manifolds - [http://arxiv.org/abs/1007.4036v2] Symplectic reduction of quasi-morphisms and quasi-states - [http://arxiv.org/abs/1612.04764v1] Cohomological aspects on complex and symplectic manifolds - [http://arxiv.org/abs/1511.09051v3] On automorphism groups of affine surfaces - [http://arxiv.org/abs/1002.3099v3] Tamed Symplectic forms and SKT metrics - [http://arxiv.org/abs/2012.03134v On the minimal symplectic area of Lagrangians - [http://arxiv.org/abs/1007.3281v3] Altering symplectic manifolds by homologous recombination

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def communication_complexity(f, M):
        return len(f) * len(M)

    def generate_symplectic_manifold(n):
        # Simplified generation of a symplectic manifold with n elements
        return list(range(1, n + 1))

    def evaluate_polynomial(poly, M):
        result = 0
        for term in poly:
            product = 1
            for var in term:
                product *= M[var - 1]
            result += product
        return result

    def generate_random_poly(n):
```

```
        # Generate a random polynomial of degree n
        return [random.randint(1, 5) for _ in range(n)]

    instances_tested = 0
    total_comm_complexity = 0
    n_max = 0

    for n in {5, 10, 15, 20, 30, 40}:
        M = generate_symplectic_manifold(n)
        f = generate_random_poly(n)
        comm_complexity = communication_complexity(f, M)
        total_comm_complexity += comm_complexity
        instances_tested += len(f)
        n_max = max(n_max, n)

    mean_comm_complexity = total_comm_complexity / instances_tested

    # Check if the mean communication complexity is within ±10% of O(n^2)
    conjecture_holds = abs(mean_comm_complexity - n_max**2) <= 0.1 * n_max**2
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "communication_complexity",
        "metric_value": mean_comm_complexity,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_comm_complexity = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_comm_complexity} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_comm_complexity} std=0.0 support_fraction={support_fr
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ndefined'}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 27.083333333333332, 'instances_to
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 27.083333333333332, 'instances_to
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 27.083333333333332, 'instances_to
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 27.083333333333332, 'instances_to
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 27.083333333333332, 'instances_to
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 27.083333333333332, 'instances_to
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 27.083333333333332, 'instances_to
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 27.083333333333332, 'instances_to
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 27.083333333333332, 'instances_to

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_216d8ab3.py", line 86, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                              ^^^^^^^
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data that could confirm or falsify the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14361
2	propose	ollama_remote	glm4:latest	0	0	18325
3	preregistration	ollama_remote	glm4:latest	0	0	13906
4	novelty	ollama_remote	glm4:latest	0	0	8265
5	novelty	ollama_remote	glm4:latest	0	0	19867
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20014
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24687
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9184
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20772
10	judge	ollama_remote	glm4:latest	0	0	9860

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 159241 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b0d16ff55b01.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b0d16ff55b01.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b0d16ff55b01.tar.gz` (if generated)